Module 2

1. Python Functions

A **function** is a block of code used to perform a specific task.

```
def greet():
    print("Hello, Welcome to Python!")

greet()
```

Output

Hello, Welcome to Python!

Parameters

```
def add(a, b):
    print(a + b)

add(10, 20)
```

Output
30

Return Value

`return` sends a result back from the function.

```
def multiply(a, b):
    return a * b

result = multiply(5, 4)
print(result)
```

Output
20

2. Lambda Functions

A **lambda function** is a small function written in one line.

```
square = lambda x: x * x

print(square(5))
```

Output

25

3. List Comprehension

List comprehension is a simple way to create a list.

Normal method:

```
numbers = [i for i in range(1, 6)]

print(numbers)
```

Output

[1, 2, 3, 4, 5]

4. File Handling

File handling is used to **create, read, write, and modify files**.

```
file = open("student.txt", "w")

file.write("Vardhini\n")
file.write("AIML Student")

file.close()
```

Output

Vardhini
AIML Student

5. CSV Files

CSV means **Comma-Separated Values**. It is commonly used to store tabular data.

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["ID", "Name", "Marks"])
    writer.writerow([1, "Vardhini", 90])
    writer.writerow([2, "Rahul", 85])

print("Data saved successfully")
```

Output

```
ID,Name,Marks
1,Vardhini,90
2,Rahul,85
```

6. JSON Files

JSON is another format used to store data.

```
import json

student = {
    "id": 1,
    "name": "Vardhini",
    "marks": 90
}

with open("student.json", "w") as file:
```

```
    json.dump(student, file, indent=4)

print("JSON file created")
```

Output

```
{'id': 1, 'name': 'Vardhini', 'marks': 90}
```

7. Virtual Environment

A **virtual environment** creates a separate Python environment for a project.

In VS Code terminal, run:

```
python -m venv myenv
```

Activate it on Windows:

```
myenv\Scripts\activate
```

8. Exception Handling

Exception handling is used to handle errors without stopping the entire program.

```
try:
    a = 10
    b = 0
    print(a / b)

except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output

Cannot divide by zero

9. Debugging

Debugging means finding and fixing errors in a program.

```
a = 10
b = 20

print("Value of a:", a)
print("Value of b:", b)

result = a + b

print("Result:", result)
```

10. Student Record Management System

This project combines **functions, lists, dictionaries, CSV/JSON, CRUD, and exception handling**.

CRUD means:

- **C – Create** → Add student
- **R – Read** → View students
- **U – Update** → Update student
- **D – Delete** → Delete student

```
import csv
import os

FILE_NAME = "students.csv"
```

```
def create_file():
    if not os.path.exists(FILE_NAME):
        with open(FILE_NAME, "w", newline="") as file:
            writer = csv.writer(file)
            writer.writerow(["ID", "Name", "Marks"])


def add_student():
    student_id = input("Enter Student ID: ")
    name = input("Enter Student Name: ")
    marks = input("Enter Marks: ")

    with open(FILE_NAME, "a", newline="") as file:
        writer = csv.writer(file)
        writer.writerow([student_id, name, marks])

    print("Student added successfully!")


def view_students():
    with open(FILE_NAME, "r") as file:
        reader = csv.reader(file)

        for row in reader:
            print(row)


def update_student():
    student_id = input("Enter Student ID to update: ")

    rows = []

    with open(FILE_NAME, "r") as file:
        reader = csv.reader(file)
```

```
        rows = list(reader)

    found = False

    for row in rows:
        if row[0] == student_id:
            row[1] = input("Enter new name: ")
            row[2] = input("Enter new marks: ")
            found = True

    with open(FILE_NAME, "w", newline="") as file:
        writer = csv.writer(file)
        writer.writerows(rows)

    if found:
        print("Student updated successfully!")
    else:
        print("Student not found!")


def delete_student():
    student_id = input("Enter Student ID to delete: ")

    rows = []

    with open(FILE_NAME, "r") as file:
        reader = csv.reader(file)
        rows = list(reader)

    new_rows = [row for row in rows if row[0] != student_id]

    with open(FILE_NAME, "w", newline="") as file:
        writer = csv.writer(file)
        writer.writerows(new_rows)
```

```
    print("Student deleted successfully!")


create_file()

while True:

    print("\n--- Student Record Management System ---")
    print("1. Add Student")
    print("2. View Students")
    print("3. Update Student")
    print("4. Delete Student")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()

    elif choice == "2":
        view_students()

    elif choice == "3":
        update_student()

    elif choice == "4":
        delete_student()

    elif choice == "5":
        print("Program ended.")
        break

    else:
        print("Invalid choice!")
```

Output

--- Student Record Management System ---
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter your choice: 1

Enter Student ID: 101
Enter Student Name: Vardhini
Enter Marks: 90

Student added successfully!